



SQL QUERY PRACTICE SET

20 Medium-to-Hard Questions - Basics, Aggregation, Joins and Subqueries

Database: Use the same **departments**, **employees**, and **projects** tables throughout. Questions progress from strong SQL fundamentals to joins and interview-level subqueries. **Questions only - no solutions are included.**

1. Table Details

departments

dept_id	dept_name	location
10	Engineering	Chennai
20	Sales	Hyderabad
30	HR	Bengaluru
40	Research	Pune
50	Support	Chennai

employees

emp_id	emp_name	dept_id	salary	manager_id
101	Arun	10	50000	103
102	Bala	20	45000	106
103	Charan	10	80000	NULL
104	Divya	30	40000	108
105	Ezhil	10	65000	103
106	Farah	20	75000	NULL
107	Gokul	40	55000	109
108	Harini	30	70000	NULL
109	Irfan	40	90000	NULL
110	Janani	10	60000	103

projects

project_id	project_name	dept_id	budget
1	Cloud Migration	10	500000
2	Mobile App	10	300000
3	CRM Upgrade	20	250000
4	Recruitment AI	30	200000



2. Practice Questions

1. Salary Band Filtering and Ranking

Topic: Fundamentals - WHERE, BETWEEN, ORDER BY Level: Medium

Display **emp_id**, **emp_name**, **salary** for employees whose salary is between 50,000 and 80,000 inclusive. Sort the result by salary descending; when salaries are equal, sort by emp_name ascending.

Expected output columns: emp_id, emp_name, salary

2. Multi-Condition Employee Search

Topic: WHERE - AND / OR / IN / NULL Level: Medium

Display employees who either belong to department 10 with salary above 55,000, or belong to department 20 with salary at least 70,000. Exclude employees whose manager_id is NULL.

Expected output columns: emp_id, emp_name, dept_id, salary, manager_id

3. Pattern Matching Challenge

Topic: LIKE - String Filtering Level: Medium

Find employees whose names contain the letter **a** anywhere after the first character and whose names do not end with **n**. Sort alphabetically.

Expected output columns: emp_id, emp_name

4. Conditional Salary Classification

Topic: CASE - Computed Columns Level: Medium

Display every employee with a derived salary_band: **High** for salary $\geq$ 75,000, **Medium** for 55,000-74,999, and **Standard** otherwise. Within each band, show higher salaries first.

Expected output columns: emp_name, salary, salary_band

5. Department Salary Statistics

Topic: GROUP BY - Aggregates Level: Medium

For each dept_id that has employees, display employee count, minimum salary, maximum salary, average salary, and total salary. Show departments with the highest average salary first.

Expected output columns: dept_id, employee_count, min_salary, max_salary, avg_salary, total_salary

6. Strong Departments Only

Topic: GROUP BY + HAVING Level: Medium-Hard

Return departments from the employees table that have at least two employees and an average salary greater than 55,000.

Expected output columns: dept_id, employee_count, avg_salary

7. Project Budget Analysis

Topic: GROUP BY + HAVING - Projects Level: Medium-Hard

For each department represented in projects, calculate project count and total budget. Return only departments whose total project budget is at least 300,000.

Expected output columns: dept_id, project_count, total_budget



8. Employee Department Report

Topic: INNER JOIN **Level:** Medium

Display each employee together with the department name and location. Include only employees whose salary is at least 60,000, ordered by department name and then salary descending.

Expected output columns: emp_id, emp_name, dept_name, location, salary

9. All Departments With Employee Counts

Topic: LEFT JOIN + GROUP BY **Level:** Medium-Hard

Display every department, including departments with no employees, together with the number of employees in that department. Departments with zero employees must appear.

Expected output columns: dept_id, dept_name, employee_count

10. Employee and Manager Names

Topic: SELF JOIN **Level:** Medium-Hard

Display every employee who has a manager, together with the manager's name and both salaries. Sort by manager name and then employee salary descending.

Expected output columns: employee_name, employee_salary, manager_name, manager_salary

11. Department Project Coverage

Topic: LEFT JOIN - Multi-table Thinking **Level:** Medium-Hard

Display every department with its project count and total project budget. Departments without projects must still be returned with a project count of 0.

Expected output columns: dept_id, dept_name, project_count, total_budget

12. Employees in Project-Active Departments

Topic: Multi-table JOIN **Level:** Hard

Display employees who belong to departments that have at least one project. Include department name and project name. An employee may appear more than once if the department owns multiple projects.

Expected output columns: emp_name, dept_name, project_name, salary

13. Highest Paid Employee(s)

Topic: Scalar Subquery **Level:** Medium

Display all employees whose salary equals the maximum salary in the employees table. The query must correctly return all ties and must not use LIMIT.

Expected output columns: emp_id, emp_name, salary

14. Employees Above Company Average

Topic: Scalar Subquery + AVG **Level:** Medium

Display employees whose salary is greater than the overall company average. Also return salary and sort from highest to lowest.

Expected output columns: emp_id, emp_name, salary



15. Employees Above Their Department Average

Topic: Correlated Subquery **Level:** Hard

Display employees whose salary is greater than the average salary of their own department. Do not solve the department comparison using a JOIN.

Expected output columns: emp_id, emp_name, dept_id, salary

16. Highest Paid Employee(s) Per Department

Topic: Correlated Subquery + MAX **Level:** Hard

Return the highest-paid employee or employees in every department. If multiple employees tie for the maximum salary within a department, all must be returned.

Expected output columns: emp_id, emp_name, dept_id, salary

17. Departments Without Projects

Topic: NOT EXISTS **Level:** Medium-Hard

Display departments for which no row exists in the projects table. Use a subquery-based existence check.

Expected output columns: dept_id, dept_name, location

18. Employees Earning More Than Every HR Employee

Topic: ALL + Nested Subquery **Level:** Hard

Display employees whose salary is greater than the salary of every employee in the HR department. Obtain the HR dept_id from the departments table rather than hard-coding it.

Expected output columns: emp_id, emp_name, salary

19. Employees in Above-Average Departments

Topic: Nested Subqueries **Level:** Hard

Display employees who belong to a department whose average employee salary is greater than the overall company average salary. Use subqueries to identify the qualifying departments.

Expected output columns: emp_id, emp_name, dept_id, salary

20. Employee(s) Closest to Company Average Salary

Topic: Advanced Nested Subqueries **Level:** Hard

Find the employee or employees whose salary has the smallest absolute difference from the overall company average salary. Do not use ORDER BY ... LIMIT to choose the closest row. Ties must be preserved.

Expected output columns: emp_id, emp_name, salary



3. Coverage Map

Questions	Coverage
1-4	SELECT, WHERE, BETWEEN, IN, NULL handling, LIKE, ORDER BY, CASE
5-7	COUNT, MIN, MAX, AVG, SUM, GROUP BY, HAVING
8-12	INNER JOIN, LEFT JOIN, SELF JOIN, multi-table joins, grouped joins
13-14	Scalar subqueries and aggregate subqueries
15-17	Correlated subqueries, MAX per group, NOT EXISTS
18-20	ALL, nested subqueries, group-vs-company comparisons, advanced nested aggregation

4. Student Rules

1. Use MySQL-compatible SQL syntax.
2. Do not hard-code department IDs when a question explicitly asks you to derive the department by name.
3. Preserve ties in maximum, minimum, and closest-value questions.
4. For Questions 15-20, use the requested subquery approach rather than replacing it with a JOIN or LIMIT shortcut.
5. Aliases are encouraged where they improve readability.
6. Before executing a complex query, verify the intermediate result: grouped result, joined result, or inner subquery result.